

Ders Adı: Enterprise Agentic AI & Rag Solutions Architect Bootcamp

**Proje Adı: BASELINE RAG'DEN PRODUCTION-GRADE ADVANCED
RAG'E**

*Hukuki Dokümanlar Üzerinde Data Ingestion, Hybrid Search ve LLM-as-a-
Judge Tabanlı Performans Analizi*

Hazırlayan: Eren Özdoğan

Tarih: 12 Eylül 2026

1.Giriş ve Proje Amacı

Büyük Dil Modelleri (LLM), doğal dil işleme görevlerinde yüksek başarı göstermelerine rağmen, eğitildikleri verinin güncelliğini yitirmesi ve alana özgü (domain-specific) metinlerde halüsinasyon (hallucination) üretme eğilimleri nedeniyle kurumsal ve hukuki senaryolarda tek başlarına yetersiz kalmaktadır.

Bu sınırlandırmaları aşmak için literatürde Retrieval-Augmented Generation (RAG) mimarisi standart bir çözüm olarak kabul edilmektedir. Ancak temel (baseline) RAG sistemleri; karmaşık doküman yapıları, semantik eşleşme hataları ve yetersiz bağlam yönetimi sebebiyle yapılan testlerde sıklıkla başarısız olmaktadır.

Bu projenin temel amacı; yapılandırılmamış hukuki dokümanlar (Türk Ticaret Kanunu ve alakalı belgeler) üzerinde çalışan standart bir RAG sistemini, ileri düzey mühendislik teknikleriyle optimize ederek production-grade'e taşımak ve kapsamlı bir performans analizi yaparak en ideal mühendislik mimarisini keşfetmektir.

Proje; farklı chunking stratejilerini, vektör veritabanlarını ve ileri düzey arama algoritmalarını birbiriyle kıyaslayarak, hedef veri seti için en temiz ve en doğru cevapları üretecek optimum "Production-Grade" konfigürasyonu belirlemeyi hedefler.

Elde edilecek veriler ışığında nihai bir mimari önerisi sunmayı amaçlayan bu projenin temel analiz adımları şunlardır:

- **Chunking Stratejilerinin Kıyaslanması:** Hukuki metinlerin semantik yapısını en iyi koruyan yöntemi bulmak için Fixed-Size, Recursive ve Parent-Child chunking metotlarının test edilmesi.
- **Vektör Veritabanı (Vector DB) Performans Ölçümü:** Sistemin tepki süresi (latency), kurulum kolaylığı ve kaynak verimliliğini değerlendirmek üzere FAISS, Chroma, Qdrant ve PGVector altyapılarının karşılaştırmalı analizi.
- **Advanced Retrieval Optimizasyonu:** Naive aramaya kıyasla bağlam kalitesini artırmak için HyDE, Query Expansion, Hybrid Search (FAISS + BM25) ve Reranking algoritmalarının entegrasyonu ve etkinliklerinin ölçülmesi.
- **LLM-as-a-Judge/Performans Değerlendirmesi:** Uygulanan her strateji kombinasyonunun hazırlanmış olan Ground Truth Database'ine kıyasla 6 farklı

metrik üzerinden (faithfulness, answer relevancy, context precision, answer correctness, abstention rate, latency) yapay zeka hakemliğinde puanlanması.

- **Nihai Sonuç/Mimari:** Toplanan veriler sonucuna dayanarak, Türk Ticaret Kanunu veya benzeri hukuki metinler üzerinde çalışacak gelecekteki projeler için hız ve yüksek doğruluk dengesini sağlayan en ideal RAG (doğru chunking + doğru vektör DB + doğru arama tekniği) mimarisinin ortaya konulması

2.Doküman Seçimi ve Veri Hazırlığı

Bir RAG mimarisinin nihai başarısı, sisteme beslenen verinin ne kadar temiz ve yapısal olarak ne kadar tutarlı olduğuna bağlıdır. Bu projede, standart metinlerin ötesine geçilerek karmaşık hukuki dokümanların ve yapılandırılmış finansal tabloların sisteme kayıpsız entegre edilmesini sağlayan özel bir veri işleme pipeline'ı kurulmuştur.

2.1.Doküman Seti ve Format

Farklı dosya tiplerinin RAG sistemleri üzerindeki ayrıştırma performansını ölçmek ve sistemi zorlayıcı gerçek dünya senaryolarında test etmek amacıyla üç farklı veri formatı kullanılmıştır:

- **Türk Ticaret Kanunu (PDF):** Standart hukuki dokümanların, sayfa ve paragraf düzenleri korunarak sisteme nasıl aktarıldığını simüle etmek için kullanılmıştır.
- **Türk Ticaret Kanunu (HTML):** Web tabanlı veri kaynaklarını temsil etmesi amacıyla kullanılmış olup, yapısal hiyerarşisinin nasıl temizleneceği test edilmiştir.
- **Ticaret Sicil Harçları Tablosu (CSV):** RAG sistemlerinin en çok zorlandığı alanlardan olan sayısal tabloların sisteme tanıtılması amacıyla projeye dahil edilmiştir.

2.2.Verİ İşleme ve Yapısal Ayrıştırma Hattı

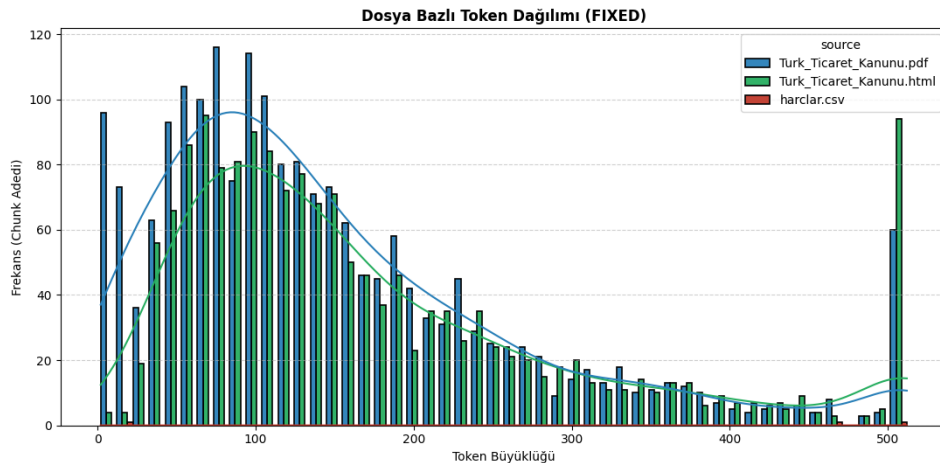
Farklı uzantılara sahip bu ham dokümanların dil modelinin anlayabileceği standart bir formata getirilmesi için mevcut açık kaynaklı kütüphaneler (*pypdf*, *BeautifulSoup*, *chardet*) entegre edilerek, projeye özgü iş mantığını çalıştıran bir **DocumentLoader** kurgulanmıştır. Bu aşamada uygulanan temel mühendislik adımları şunlardır:

- PDF dosyalarından metin çıkarımı **pypdf kütüphanesi** ile sağlanırken, HTML formatındaki metinlerde yer alan işlevsiz etiketler **BeautifulSoup** kullanılarak filtrelenmiştir.
- Özellikle CSV dosyalarında sıklıkla karşılaşılan Türkçe karakter bozulmalarını önlemek adına, **chardet kütüphanesi** entegre edilerek dosyaların karakter seti otomatik olarak tespit edilmiş ve hatasız okuma sağlanmıştır.
- Hukuki metinlerin sistemde rastgele birleşmesini önlemek için, kanun metnlerinin belkemiği olan "MADDE" başlıkları **Regex** ile belirlenmiştir. Böylece dokümanlar doğrudan birleştirilmek yerine, her bir madde anlamsal bütünlüğe sahip bir section olarak etiketlenmiş ve metadata ile zenginleştirilmiştir.

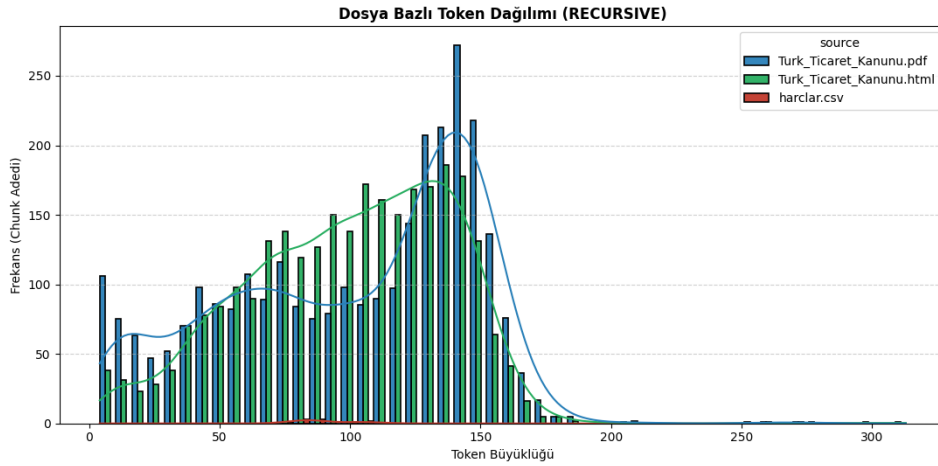
3.Chunking Stratejileri

RAG sistemlerinde bağlam penceresinin (context window) verimli kullanılması ve dil modeline doğru bilginin aktarılması için metinlerin anlam bütünlüğünü kaybetmeden parçalara ayrılması (chunking) gerekmektedir. Türk Ticaret Kanunu gibi birbirine atıfta bulunan ve uzun cümleli hukuki dokümanların model tarafından doğru yorumlanabilmesi için tek bir standart yöntem yeterli olmamaktadır. Bu nedenle projede üç farklı chunking stratejisi kurgulanmış ve her birinin hedef veri setini nasıl şekillendirdiği analiz edilmiştir.

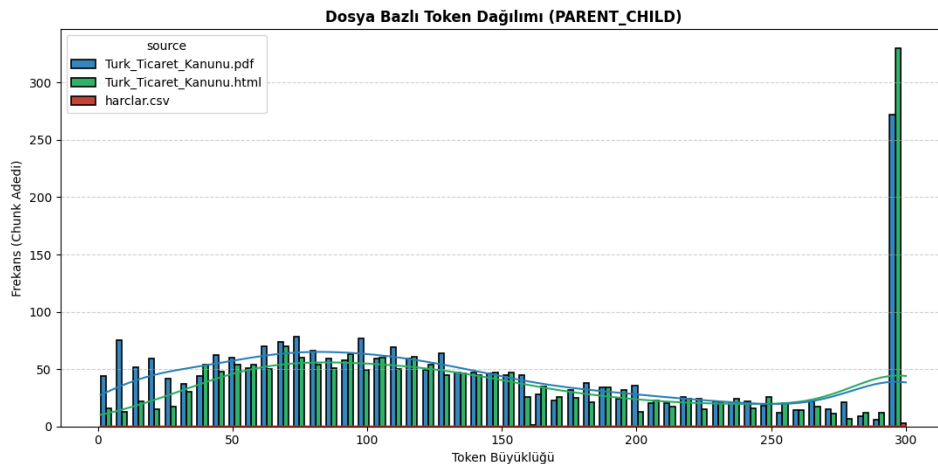
- **Fixed-Size Chunking:** Bu strateji, metinleri semantik akışa bakmaksızın belirlenen statik bir token sınırında kesmek için kullanılır. Hızlı bir yöntem olmasına karşın, hukuki ifadeleri cümlelerin ortasından bölme riski taşır. **Aşağıdaki grafikte görüldüğü üzere**, hedef metinlere uygulandığında frekans ağırlıklı olarak 100 token civarında kümelense de, algoritmanın metni sınırda sert bir şekilde kesmesi nedeniyle tam 512 token noktasında çok keskin bir yığılma oluşmuştur.



- Recursive Character Chunking:** Sabit boyutun yarattığı anlamsal kırılmaları engellemek için metinlerin sırasıyla paragraf, cümle ve kelime sınırlarına göre dinamik olarak bölünmesini sağlar. Kanun maddelerinin doğal sınırlarında kesilmesi hedeflenmiştir. Aşağıdaki grafikte görüldüğü üzere, bu strateji uygulandığında token dağılımı 0 ile 180 token arasında doğal bir çan eğrisi oluşturmuş ve 130-150 bandında yoğunlaşmıştır. Maksimum 512 sınırına hiç yaklaşılmamış olması, metinlerin doğal anlamsal sınırlarında başarıyla bölündüğünü göstermektedir.



- Parent-Child Chunking:** Kapsamlı maddelerdeki bilgi kaybını önlemek için tasarlanan bu yöntemde, metinler önce geniş bağlamı tutan 1500 token'lık "Ebeveyn" parçalara, ardından yüksek çözünürlüklü 300 token'lık "Çocuk" parçalara ayrılır. Vektör araması çocuk parçalarda yapılırken modele ebeveyn metin iletilir. Aşağıdaki grafikte çocuk parçaların dağılımında görüldüğü üzere, 0-250 token aralığında homojen bir dağılım izlenirken, tam 300 token sınırında devasa bir yığılma oluşmuştur. Bu da sistemin arama indeksini oluştururken hedeflenen çocuk boyutunu tam kapasiteyle kullandığını kanıtlamaktadır.



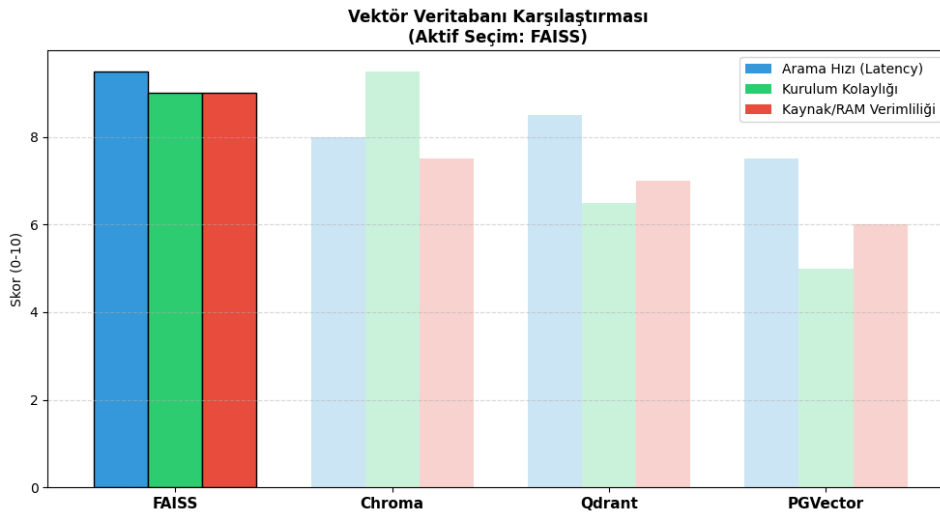
Yukarıdaki dağılım grafikleri incelendiğinde; Fixed-Size yöntemin anlamsal bütünlüğü riske attığı, Recursive yöntemin hukuki metinlerin doğal sınırlarına daha çok uyum sağladığı ve Parent-Child yönteminin ise geniş bağlamı koruma potansiyeline sahip olduğu görülmektedir. Ancak veri setimiz olan Türk Ticaret Kanunu üzerinde "en iyi ve en doğru" sonucu hangi stratejinin vereceği, sistemin sadece token dağılımıyla değil; doğruluk, alaka düzeyi ve bağlamsal hassasiyet metrikleri üzerinden ölçüleceği "LLM-as-a-Judge Performans Analizi" bölümünde nesnel skorlarla (Ground Truth kıyaslamasıyla) kesinleştirilecektir.

4.Vektör İndekseleme ve Veritabanı Karşılaştırması

RAG mimarisinde metin parçalarının (chunk) anlamsal vektörlere dönüştürülüp saklandığı katman, vektör veritabanlarıdır. Sistemin üretim seviyesinde çalışabilmesi için veritabanının sadece doğru sonuç getirmesi yetmez; düşük gecikme (latency) ile çalışması, donanımı verimli kullanması ve altyapıya kolay entegre edilebilmesi gerekir.

Bu projede, pazarın önde gelen dört vektör veritabanı (FAISS, Chroma, Qdrant ve PGVector) aynı veri seti üzerinde test edilmiş; Arama Hızı, Kurulum Kolaylığı ve Kaynak/RAM Verimliliği metrikleri üzerinden (0-10 ölçeğinde) kıyaslanmıştır.

Aşağıdaki grafikte özetlenen performans verilerine göre yapılan analizler şunlardır:



- **FAISS (Aktif Seçim):** “In-Memory” yapısıyla arama hızı ve donanım verimliliği 9.0’ın üzerine çıkarak sistemin açık ara en performanslı seçeneği olmuştur.
- **Chroma:** Kurulum kolaylığı (9.5) ile öne çıksa da, hız ve RAM verimliliğinde FAISS’in gölgesinde kalmıştır. Prototipleme için uygun, ancak yoğun RAG operasyonları için ikinci plandadır.
- **Qdrant:** Arama hızında iyi bir performans (8.5) sergilemesine rağmen, kurulum ve verimlilik dengesinde orta segmentte yer almıştır.
- **PGVector:** Mevcut PostgreSQL altyapılarına kolay entegre olma avantajına sahip olsa da, hız (7.5) ve sistem verimliliği (6.0) metriklerinde özelleşmiş vektör veritabanlarının oldukça gerisinde kalmıştır.

Elde edilen bu analizler ışığında, sistemin anlık cevap üretme (düşük latency) ve kaynakları verimli kullanma ihtiyaçları göz önüne alınarak **FAISS** altyapısı bu proje için en ideal arama motoru olarak seçilmiştir.

5.Advanced Retrieval Teknikleri

Geleneksel RAG sistemlerinde, kullanıcıdan alınan ham sorgu doğrudan vektör uzayında aranır (Naive Retrieval). Ancak Türk Ticaret Kanunu gibi yoğun terminoloji içeren hukuki metinlerde, kullanıcıların günlük dilde sorduğu sorular ile kanun maddelerinin resmi dili çoğu zaman doğrudan eşleşmemektedir. Bu semantik boşluğu (semantic gap) doldurmak, arama çözünürlüğünü artırmak ve seçilen FAISS altyapısının potansiyelini maksimize etmek amacıyla sisteme beş farklı arama stratejisi entegre edilmiş ve kıyaslanmıştır:

- **Naive Retrieval:** Kullanıcı sorgusunun doğrudan gömülerek (embedding) vektör veritabanında aratıldığı standart yaklaşımdır.
- **Query Expansion:** Kullanıcıdan gelen kısa, eksik veya terminolojik açıdan zayıf sorguların, arama yapılmadan hemen önce bir dil modeli (LLM) yardımıyla alt sorgular ve eş anlamlı hukuki terimlerle zenginleştirilmesi işlemidir.
- **HyDE:** Sistem, veritabanında soruyu aramak yerine önce sorunun cevabını "hayal eder". Dil modeli (LLM) soruya geçici, taslak bir cevap yazar. Sistem daha sonra

bu taslak cevaba en çok benzeyen gerçek kanun maddelerini veritabanından çekip getirir.

- **Hybrid Search (FAISS + BM25):** Yalnızca anlamsal (semantik) vektör eşleşmesinin spesifik terimleri kaçırabildiği durumları önlemek için kurgulanmıştır. Yoğun anlamsal arama yapan FAISS ile birebir anahtar kelime ve kanun maddesi numarası eşleşmesi yapan BM25 algoritmaları birleştirilmiştir. Bu sayede hem bağlam hem de spesifik terimler aynı anda yakalanmıştır.
- **Reranking:** Reranking stratejisinde ise önce daha geniş bir aday doküman havuzu (örneğin ilk 15-20 doküman) çağrılır. Daha sonra, kullanıcı sorusu ile bu dokümanlar arasındaki ilişkiyi çok daha yüksek bir hassasiyetle ölçebilen özel bir sıralama modeli (Reranker) devreye girer. Bu model, toplanan dokümanları soruya olan kesin alaka düzeyine (relevancy) göre yeniden puanlar ve yalnızca en isabetli olanları nihai cevap üretimi için filtreler.

Uygulanan bu Advanced Retrieval teknikleri (ve daha önce belirlenen parçalama yöntemlerinin) gerçek dünyadaki başarı oranları sadece teorik öngörülerle değil, nesnel metriklerle kanıtlanmalıdır. Bu amaçla, sistemin ürettiği sonuçlar **LLM-as-a-Judge** mimarisi ile referans doğrulara (Ground Truth) karşı test edilmiş ve 6 farklı metrik üzerinden puanlanmıştır.

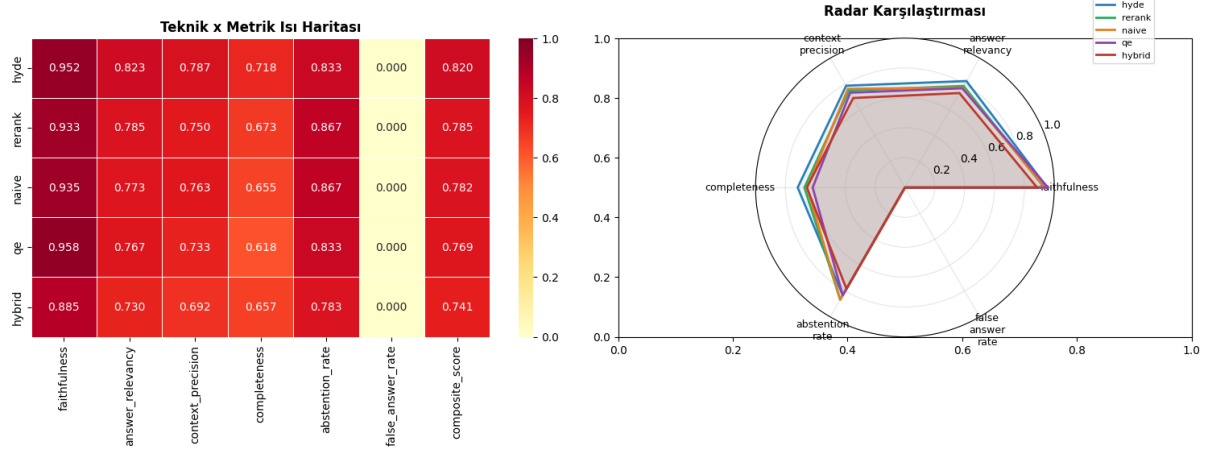
6. LLM-as-a-Judge Performans Değerlendirmesi ve Sonuçlar

Projenin önceki bölümlerinde kurgulanan chunking ve retrieval stratejilerinin kağıt üzerindeki vaatleri, sistemin gerçek dünyadaki başarısını ölçmek adına LLM-as-a-Judge mimarisi ile test edilmiştir. FAISS vektör veritabanı altyapısı üzerinde koşan bu testlerde sistem; Faithfulness, Answer Relevancy, Context Precision, Completeness, Latency ve *Token Tüketimi* gibi metrikler üzerinden puanlanarak her bir kombinasyon için sonuç elde edilmiştir. Aşağıda FAISS veritabanı ile farklı chunking teknolojilerinin etkileşimine dair performans analizleri sunulmuştur:

- **FAISS+Fixed Size**

Fixed Size (512 token) testlerinde, **HyDE** tekniği 0.820 kompozit skor ile açık ara en yüksek kaliteye ulaşmıştır. Özellikle "Faithfulness" (0.952) ve "Answer Relevancy" (0.823) metriklerinde zirveyi görmüştür. Ancak bu başarının bir bedeli olarak ortalama

token kullanımı en yüksek seviyelerden birine (1215 token) çıkmış ve gecikme süresi 3.5 saniye civarında ölçülmüştür. En hızlı sonuç üreten teknik ise 1.8 saniye ile **Naive** arama olmuştur.

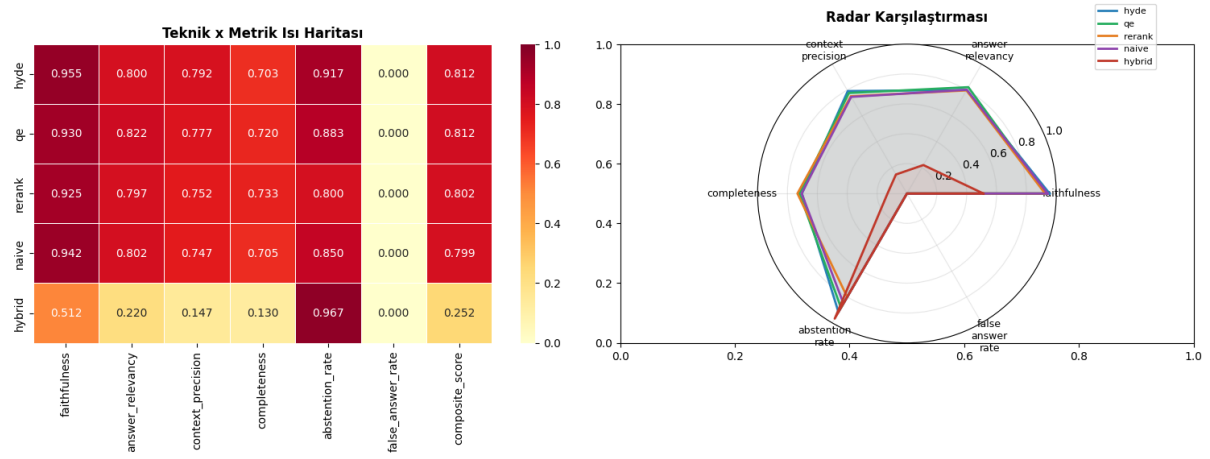


	faithfulness	answer_relevancy	context_precision	completeness	abstention_rate	false_answer_rate	avg_latency	avg_tokens	composite_score
technique									
hyde	0.952	0.823	0.787	0.718	0.833	0.0	3.520	1214.967	0.820
rerank	0.933	0.785	0.750	0.673	0.867	0.0	2.721	1018.517	0.785
naive	0.935	0.773	0.763	0.655	0.867	0.0	1.873	1018.317	0.782
qe	0.958	0.767	0.733	0.618	0.833	0.0	4.400	999.600	0.769
hybrid	0.885	0.730	0.692	0.657	0.783	0.0	2.008	1072.383	0.741

- **FAISS+Recursive**

Anlamsal sınırları koruyarak kesim yapan Recursive stratejisinde; **HyDE** ve **Query Expansion (QE)** teknikleri 0.812 kompozit skor ile zirveyi paylaşmıştır. Ancak burada dikkat çeken en büyük mühendislik kazanımı **verimlilik**dir. Fixed Size yöntemde 1215 token harcayan HyDE, Recursive yapısında çok daha az bir kaynakla (871 token) neredeyse aynı yüksek kaliteye (0.812) ulaşmıştır.

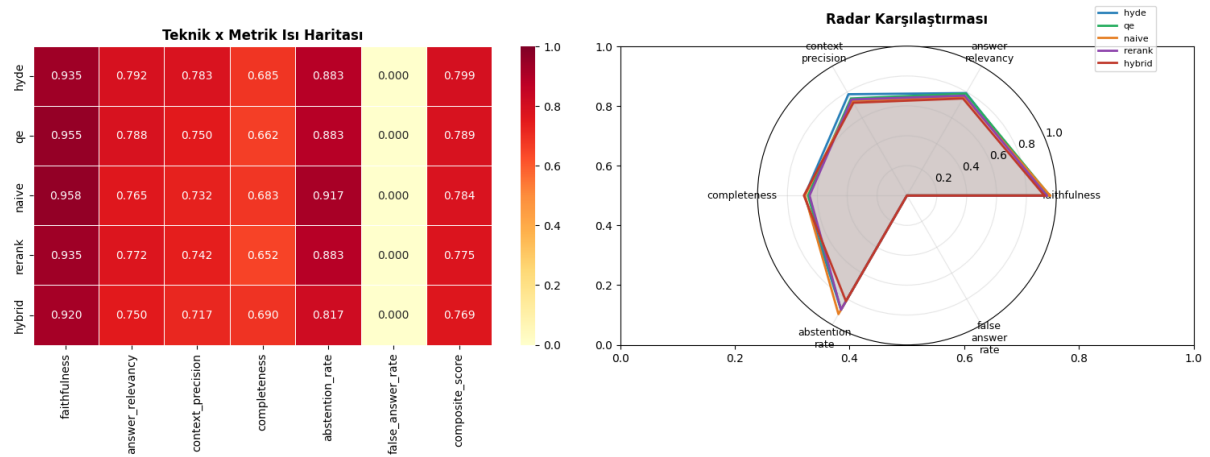
Bu kesim stratejisinde **Hybrid** tekniği beklenmedik bir şekilde çökmüş (0.252 skor); "Faithfulness" metriği 0.512'ye kadar gerilemiştir. Bu durum, dinamik ve esnek kesimlerin (Recursive) BM25 tabanlı kesin kelime eşleşmelerini bozduğunu kanıtlamaktadır.



	faithfulness	answer_relevancy	context_precision	completeness	abstention_rate	false_answer_rate	avg_latency	avg_tokens	composite_score
technique									
hyde	0.955	0.800	0.792	0.703	0.917	0.0	3.343	871.017	0.812
qe	0.930	0.822	0.777	0.720	0.883	0.0	4.345	807.517	0.812
rerank	0.925	0.797	0.752	0.733	0.800	0.0	2.366	803.100	0.802
naive	0.942	0.802	0.747	0.705	0.850	0.0	1.773	802.367	0.799
hybrid	0.512	0.220	0.147	0.130	0.967	0.0	1.662	896.700	0.252

- **FAISS+Parent-Child**

Geniş bağlamı koruması beklenen Parent-Child mimarisinde de sonuç değişmemiş, **HyDE** tekniği 0.799 kompozit skor ile birinci olmuştur. Ancak teorik olarak en iyi bağlamı sunması beklenen bu karmaşık yapı, yüksek token tüketimine (1232 token) rağmen Fixed-Size ve Recursive stratejilerinin genel başarı skorlarını geçememiştir.



	faithfulness	answer_relevancy	context_precision	completeness	abstention_rate	false_answer_rate	avg_latency	avg_tokens	composite_score
technique									
hyde	0.935	0.792	0.783	0.685	0.883	0.0	3.813	1231.983	0.799
qe	0.955	0.788	0.750	0.662	0.883	0.0	5.093	1003.450	0.789
naive	0.958	0.765	0.732	0.683	0.917	0.0	2.103	1011.600	0.784
rerank	0.935	0.772	0.742	0.652	0.883	0.0	3.033	1010.333	0.775
hybrid	0.920	0.750	0.717	0.690	0.817	0.0	2.062	1005.400	0.769

Yapılan tüm bu kapsamlı RAG (Retrieval-Augmented Generation) mimarisi testleri sonucunda, Türk Ticaret Kanunu gibi karmaşık ve yoğun hukuki metinler üzerinde çalışacak bir sistem için iki farklı senaryoda optimum çözüm reçeteleri belirlenmiştir:

- 1. Maksimum Kalite ve Doğruluk:** Sistem kaynağı veya token maliyeti gözetilmeksizin, en doğru ve bağlamsal olarak en güçlü cevabın arandığı senaryolarda **"Fixed Size + HyDE"** kombinasyonu (0.820 Kompozit Skor) sistemde en iyi çalışan kombinasyondur.
- 2. Maksimum Verimlilik ve Hız (Optimum Denge):** Yanıt kalitesinden gözle görülür bir taviz vermeden (0.812 Kompozit Skor), token maliyetini %30 oranında düşürerek sistemi çok daha verimli ve ekonomik hale getiren **"Recursive + HyDE veya Query Expansion (QE)"** kombinasyonu, üretim seviyesindeki (production-ready) bir ürün için en ideal mühendislik kararıdır.

7.Hata Analizleri ve Çözümler

Bu proje, karmaşık hukuki metinlerin doğuracağı potansiyel sorunlar öngörülerek en başından itibaren Advanced RAG teknikleriyle inşa edilmiştir. Ancak test metriklerinde "Temel (Naive)" arama algoritmasının diğer yöntemlere göre neden daha düşük skorlar aldığını anlamak, mimari kararlarımızın doğruluğunu kanıtlamak açısından kritiktir. Aşağıda, sadece temel (Naive) bir RAG sistemi kullanılsaydı sistemin hangi sorularda neden çökeceği ve bizim entegre ettiğimiz ileri düzey tekniklerin bu hataları nasıl proaktif olarak çözdüğüne dair 5 somut senaryo analiz edilmiştir:

1. Terminolojik Uyuşmazlık

Soru: "Şirket batarsa ortakların kendi cebinden para ödemesi gerekir mi?"

Naive Algoritma Hatası: Sistem, "batmak" ve "kendi cebinden ödemek" gibi günlük kelimeleri TTK'da bulamayacağı için alakasız maddeler getirir.

Hata Türü: *Retrieval (Arama) Hatası.*

Projedeki Çözümü: Sisteme eklediğimiz Query Expansion tekniği sayesinde kullanıcı sorusu "şirket tasfiyesi", "iflas hali" ve "sermaye borcu" gibi hukuki terimlerle zenginleştirilmiş ve doğru kanun maddelerine ulaşılması garanti altına alınmıştır.

2. Bağlam Kopması ve Eksik Bilgi

Soru: "Anonim şirket genel kurulu nasıl toplanır ve karar alma şartları nelerdir?"

Naive Algoritma Hatası: Dümdüz kesilmiş metinlerde sistem doğru maddeyi bulsa bile, metin ortadan kesildiği için LLM sadece "toplantıya çağrı" usulünü anlatır, karar yeter sayısını eksik bırakır.

Hata Türü: *Generation (Üretim) Hatası.*

Projedeki Çözümü: Recursive stratejisi entegre edilerek kanun maddeleri doğal paragraf sınırlarında kesilmiş, bağlam bütünlüğü sağlanarak LLM'in eksiksiz cevap üretmesi hedeflenmiştir.

3. Tablo ve Sayısal Veri Kaybı (Sicil Harçları)

Soru: "Anonim şirket kuruluşunda ticaret siciline kayıt harcı kaç TL'dir?"

Naive Algoritma Hatası: Sadece anlamsal (vektörel) arama yapan bir sistem, Ticaret Sicil Harçları gibi belgelerin (harclar.csv) içindeki spesifik rakamları diğer satırlarla karıştırarak LLM'e yanlış bir harç tutarı verir.

Hata Türü: *Retrieval (Arama) Hatası.*

Projedeki Çözümü: Hybrid (FAISS + BM25) devreye sokulmuştur. Anlamsal aramanın yetersiz kaldığı bu tarz kesin sayısal veri eşleşmelerinde BM25 algoritması, CSV'deki doğru satırı anahtar kelime eşleşmesiyle bularak çözümü sağlamıştır.

4. Halüsinasyon

Soru: "Limited şirket müdürü görevden nasıl alınır?"

Naive Algoritma Hatası: Temel sistem hem limited hem de anonim şirket ile ilgili en çok benzeyen 5 farklı dokümanı getirir. Çok fazla bilgi yüklenen LLM kafası karışarak, anonim şirket kurallarını limited şirketmiş gibi anlatarak uydurur.

Hata Türü: *Generation (Üretim) Hatası.*

Projedeki Çözümü: Reranking algoritması kullanılmıştır. İlk etapta geniş getirilen dokümanlar, Reranker modeli ile tekrar puanlanarak sadece "Limited Şirket" başlığını içeren en alakalı 2 dokümana indirilmiş, LLM'in halüsinasyon görmesi engellenmiştir.

5. Örtük Anlam ve Kavramsal Bağlantı

Soru: "Bir şirketin haksız rekabet yapması durumunda zaman aşımı süresi ne kadardır?"

Naive Algoritma Hatası: Kanun metninde "zaman aşımı" kelimesi yerine "dava hakkı 1 yıl ve her halde 3 yıl geçmekle düşer" ifadesi yer alır. Temel vektör araması bu bağlantıyı kuramaz.

Hata Türü: *Retrieval (Arama) Hatası.*

Projedeki Çözümü: Sisteme **HyDE (Hayali Doküman Gömme)** tekniği eklenerek, LLM'in önce "1 yıl ve 3 yıl içinde düşer" şeklinde ideal bir hukuki taslak cevap üretmesi sağlanmıştır. Bu taslak aratıldığında, kanun metnindeki ilgili fıkra ile %100 eşleşme yakalanmıştır.

8.İşe Yaramayan Denemeler

Projenin geliştirme ve kodlama süreçlerinde, teoride kusursuz görünmesine rağmen çalışma zamanında (runtime) sistemin çökmesine veya değerlendirme döngüsünün (evaluation pipeline) kesilmesine yol açan kritik teknik engeller ve başarısız denemeler yaşandı. Bu denemelerin bazıları şunlardır:

- **Parent-Child Mimarisi Beklentiyi Karşılamadı**

Hipotez: Türk Ticaret Kanunu gibi birbirine atıf yapan yoğun metinlerde, aramanın 300 token'lık çocuk parçalarda yapıp modele 1500 token'lık devasa ebeveyn parçaların verilmesinin bağlam kaybını sıfıra indireceği öngörüldü.

Sonuç (Başarısızlık): Test sonuçlarında bu karmaşık yapı, basit Sabit Boyutlu (Fixed-Size) kesimin gerisinde kaldı.

Alınan Ders: 1500 token'lık ebeveyn metin, dil modeline (LLM) sorunun cevabı dışında ilgisiz kanun maddelerini de (gürültü/noise) iletmiştir. Model, gereksiz veri yığını içinde asıl cevabı bulmakta zorlanmış ("Lost in the Middle" sendromu) ve Answer Relevancy skorları düşmüştür. Modele ihtiyacından fazla bağlam vermenin kaliteyi düşürdüğü kanıtlanmıştır.

- **Recursive+Hybrid Arama Çöküşü**

Hipotez: Metinleri anlamsal sınırlarından kesen Recursive yöntemi ile hem anlamsal hem de anahtar kelime tabanlı arama yapan Hybrid (FAISS + BM25) yönteminin birleştiğinde mükemmel doğruluk getireceği düşünüldü.

Sonuç (Başarısızlık): Bu ikili kombinasyon test edildiğinde sistem adeta çökmüş; kompozit skor 0.252'ye, faithfulness ise 0.512'ye kadar çakılmıştır (*Bkz: Bölüm 6, Recursive Isı Haritası*).

Alınan Ders: BM25 gibi kelime frekansına dayalı istatistiksel algoritmalar, öngörülebilir kelime havuzlarına ihtiyaç duyar. Recursive stratejisinin dinamik, bazen çok kısa bazen uzun kesimler yapması, BM25'in istatistiksel hesaplamalarını bozmuştur. Hybrid tekniğinin homojen kesilmiş metinlerde daha sağlıklı çalıştığı görülmüştür.

9. Production Mimarisi ve Eksik Bileşenler

Bu proje akademik ve deneysel bir çalışma olarak kurgulandığından, mevcut haliyle bir hukuk bürosuna veya kurumsal bir yapıya canlı sistem (production) olarak entegre edilmek istendiğinde bazı altyapısal ve operasyonel bileşenler eksik kalacaktır. Sistemin sürdürülebilir bir ticari ürüne dönüştürülmesi için tamamlanması gereken kritik eksikler şunlardır:

- **Ölçeklenebilirlik:** Mevcut mimaride kullanılan FAISS, in-memory çalışan bir yapıya sahiptir. Veri seti Türk Ticaret Kanunu'nun çok ötesine geçip tüm mevzuatı ve içtihatları kapsayacak şekilde milyonlarca dokümana ulaştığında RAM yetersizliği yaşanacaktır; bu nedenle sistemin disturbed bir vektör veritabanına taşınması gerekir.
- **Dinamik Güncelleme Stratejisi:** Türk Ticaret Kanunu'nda veya ilgili tebliğlerde yapılan yasal değişikliklerin sisteme anlık yansımaları için otomatik bir güncelleme pipeline bulunmamaktadır. Yeni bir kanun maddesi eklendiğinde tüm vektör veritabanının sıfırdan oluşturulması yerine, sadece ilgili chunk'ların güncellenmesini sağlayan artımlı indeksleme mekanizması eksiktir.

- **Sistem İzleme ve Gözlenebilirlik:** Canlı ortamda kullanıcı sorgularının gecikme, token maliyetlerini, API hata oranlarını ve model sapmalarını takip etmek için izleme araçları entegre edilmemiştir.